# Frontend Integration Guide: `MagizinePublic.Api` Article Bookmarks (Phase 2)

This guide covers the integration of **Article Bookmarks ( `Tbl_ReaderBookmark` )** in `MagizinePublic.Api` ( `http://localhost:5080` / `https://localhost:7080` ).

---

## 1. Architectural Highlights

- **Seamless Anonymous & Authenticated Experience**:
- Unauthenticated visitors can freely browse articles ( `GET /api/public/article/list` , `GET /api/public/article/{slug}` ). In both endpoints, each article returns its total public `BookmarkCount` , while `IsBookmarked` defaults to `false` .
- When an authenticated reader views articles (via `reader_access_token` cookie or `Authorization: Bearer <token>` ), the backend automatically checks `dbo.Tbl_ReaderBookmark` and populates `IsBookmarked = true` for articles they have saved.
- **Idempotent Write Operations**:
- **`POST /api/public/bookmark/toggle`** : Atomically adds or removes the bookmark, returning the updated `IsBookmarked` boolean and fresh `BookmarkCount` .
- **`POST /api/public/bookmark/add`** : Idempotently creates the bookmark (no error if already bookmarked).
- **`POST /api/public/bookmark/remove`** or **`DELETE /api/public/bookmark/{articleId}`** : Idempotently deletes the bookmark (no error if already removed).
- **Relational Integrity**:
- `dbo.Tbl_ReaderBookmark` enforces a compound unique constraint on `(ReaderId, ArticleId)` .
- Cascades on delete: Deleting a reader or deleting an article automatically removes associated bookmarks.

---

## 2. Seed Data Reference (For Local Testing)

| Entity | ID / Value | Notes |
|---|---|---|
| **Test Reader** | `01JREADER0000000000000001` | Username: `sophia.reader` , Password: `Admin@123` |
| **Test Published Article** | `01JART0000000000000000001` | Slug: `the-art-of-quiet-luxury` |
| **Seeded Bookmark** | `01JBKMRK000000000000000001` | Pre-bookmarked for `sophia.reader` |

---

## 3. Bookmark API Endpoints ( `/api/public/bookmark` )

All bookmark management routes require reader authentication ( `[Authorize]` ), except `check/{articleId}` which safely supports both anonymous callers (returning `false` ) and authenticated readers.

### 3.1 Toggle Bookmark ( `POST /api/public/bookmark/toggle` )

Toggles the bookmark state for an article. Supports request body or route parameter ( `POST /toggle/{articleId}` ).

**Request**:

```
POST /api/public/bookmark/toggle HTTP/1.1
Host: localhost:5080
Authorization: Bearer <reader_access_token>
Content-Type: application/json

{
  "ArticleId": "01JART0000000000000000001"
}
```

**Response ( `200 OK` )**:

```
{
  "Data": {
    "ArticleId": "01JART0000000000000000001",
    "IsBookmarked": true,
    "BookmarkCount": 42,
    "Message": "Article bookmarked successfully."
  },
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Operation completed successfully.",
  "RespType": "Success"
}
```

---

### 3.2 Explicit Add ( `POST /api/public/bookmark/add` )

Idempotently marks an article as bookmarked.

**Request**:

```
POST /api/public/bookmark/add HTTP/1.1
Host: localhost:5080
Authorization: Bearer <reader_access_token>
Content-Type: application/json

{
  "ArticleId": "01JART0000000000000000001"
}
```

**Response ( `200 OK` )**:

```
{
  "Data": {
    "ArticleId": "01JART0000000000000000001",
    "IsBookmarked": true,
    "BookmarkCount": 42,
    "Message": "Article bookmarked successfully."
  },
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Operation completed successfully.",
  "RespType": "Success"
}
```

---

### 3.3 Explicit Remove ( `POST /api/public/bookmark/remove` or `DELETE /api/public/bookmark/{articleId}` )

Idempotently removes an article from bookmarks.

**Request**:

```
POST /api/public/bookmark/remove HTTP/1.1
Host: localhost:5080
Authorization: Bearer <reader_access_token>
Content-Type: application/json

{
  "ArticleId": "01JART0000000000000000001"
}
```

**Response ( `200 OK` )**:

```
{
  "Data": {
    "ArticleId": "01JART0000000000000000001",
    "IsBookmarked": false,
    "BookmarkCount": 41,
    "Message": "Article removed from bookmarks."
  },
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Operation completed successfully.",
  "RespType": "Success"
}
```

---

### 3.4 Check Bookmark Status ( `GET /api/public/bookmark/check/{articleId}` )

Queries whether the current reader has bookmarked the article. Allows anonymous calls (returns `IsBookmarked: false` without 401 error).

**Request**:

```
GET /api/public/bookmark/check/01JART0000000000000000001 HTTP/1.1
Host: localhost:5080
Authorization: Bearer <reader_access_token>
```

**Response ( `200 OK` )**:

```
{
  "Data": {
    "ArticleId": "01JART0000000000000000001",
    "IsBookmarked": true
  },
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Operation completed successfully.",
  "RespType": "Success"
}
```

---

### 3.5 My Bookmarks Paged List ( `GET /api/public/bookmark/my-bookmarks` )

Retrieves a paginated list of all Published articles bookmarked by the authenticated reader, ordered by `BookmarkedAt` descending.

**Query Parameters**:

- `Page` : 1-based page number (default: `1` )
- `PageSize` : Number of items per page (default: `10` , max: `100` )
- `ArticleCategoryId` : (Optional) Filter by category ULID
- `SearchKeyword` : (Optional) Filter by title/subtitle substring

**Request**:

```
GET /api/public/bookmark/my-bookmarks?Page=1&PageSize=10 HTTP/1.1
Host: localhost:5080
Authorization: Bearer <reader_access_token>
```

**Response ( `200 OK` )**:

```
{
  "Data": {
    "Items": [
      {
        "ReaderBookmarkId": "01M4V1EA88AP7MFY7JKCBPDJHM",
        "ArticleId": "01JART00000000000000000001",
        "BookmarkedAt": "2026-10-13T16:27:14.331+00:00",
        "Article": {
          "ArticleId": "01JART00000000000000000001",
          "Title": "The Art of Quiet Luxury",
          "SubTitle": "Why subtle craftsmanship outlasts bold logos in the modern wardrobe.",
          "Slug": "the-art-of-quiet-luxury",
          "PhotoUrl": "https://images.unsplash.com/photo-1490481651871-ab68de25d43d",
          "ArticleCategoryId": "01JARTCAT00000000000000001",
          "CategoryName": "Beauty",
          "AuthorId": "01JAUTHOR0000000000000001",
          "AuthorName": "Elena Rostova",
          "Status": "Published",
          "PublishedAt": "2026-03-01T10:00:00+00:00",
          "IsFeatured": true,
          "IsSpotlight": false,
          "IsBookmarked": true,
          "BookmarkCount": 42,
          "Photos": [
            {
              "ArticlePhotoId": "01JARTPHT00000000000000001",
              "ArticleId": "01JART00000000000000000001",
              "PhotoUrl": "https://images.unsplash.com/photo-1490481651871-ab68de25d43d",
              "Caption": "Minimalist wool coat styled with neutral cashmere.",
              "Credit": "Photo by Elena Rostova",
              "SortOrder": 1
            }
          ]
        }
      }
    ],
    "Page": 1,
    "PageSize": 10,
    "TotalCount": 1,
    "TotalPages": 1,
    "HasPrevious": false,
    "HasNext": false
  },
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Operation completed successfully.",
  "RespType": "Success"
}
```

## 4. TypeScript Models

```
export interface BookmarkToggleResModel {
  ArticleId: string;
  IsBookmarked: boolean;
  BookmarkCount: number;
  Message: string;
}

export interface BookmarkCheckResModel {
  ArticleId: string;
  IsBookmarked: boolean;
}

export interface BookmarkedArticleResModel {
  ReaderBookmarkId: string;
  ArticleId: string;
  BookmarkedAt: string;
  Article: ArticleResModel;
}

export interface ArticleResModel {
  ArticleId: string;
  Title: string;
  SubTitle?: string;
  Slug: string;
  Body: string;
  PhotoUrl?: string;
  PhotoCaption?: string;
  PhotoCredit?: string;
  ArticleCategoryId: string;
  CategoryName: string;
  AuthorId: string;
  AuthorName: string;
  Status: string;
  PublishedAt?: string;
  IsFeatured: boolean;
  IsSpotlight: boolean;
  IsBookmarked: boolean;
  BookmarkCount: number;
  Photos: ArticlePhotoResModel[];
  CreatedAt: string;
  UpdatedAt?: string;
}
```

## 5. Frontend React Component Example (Optimistic Bookmark Button)

```
import React, { useState } from 'react';
import axios from 'axios';

interface BookmarkButtonProps {
  articleId: string;
  initialIsBookmarked: boolean;
  initialBookmarkCount: number;
  onRequireAuth?: () => void;
}

export const BookmarkButton: React.FC<BookmarkButtonProps> = ({
  articleId,
  initialIsBookmarked,
  initialBookmarkCount,
  onRequireAuth
}) => {
  const [isBookmarked, setIsBookmarked] = useState(initialIsBookmarked);
  const [count, setCount] = useState(initialBookmarkCount);
  const [loading, setLoading] = useState(false);

  const handleToggle = async () => {
    if (loading) return;

    // Optimistic UI update
    const previousBookmarked = isBookmarked;
    const previousCount = count;
    const nextBookmarked = !previousBookmarked;
    const nextCount = nextBookmarked ? previousCount + 1 : Math.max(0, previousCount - 1);

    setIsBookmarked(nextBookmarked);
    setCount(nextCount);
    setLoading(true);

    try {
      const response = await axios.post(
        'http://localhost:5080/api/public/bookmark/toggle',
        { ArticleId: articleId },
        { withCredentials: true } // Passes reader_access_token cookie automatically
      );

      if (response.data.IsSuccess) {
        setIsBookmarked(response.data.Data.IsBookmarked);
        setCount(response.data.Data.BookmarkCount);
      } else {
        // Rollback
        setIsBookmarked(previousBookmarked);
        setCount(previousCount);
      }
    } catch (error: any) {
      // Rollback on error
      setIsBookmarked(previousBookmarked);
      setCount(previousCount);

      if (error?.response?.status === 401 && onRequireAuth) {
        onRequireAuth(); // Trigger Login/Register modal for guest readers
      }
    } finally {
      setLoading(false);
    }
```

```
  };

  return (
    <button
      onClick={handleToggle}
      disabled={loading}
      className={`bookmark-btn ${isBookmarked ? 'bookmarked' : ''}`}
      aria-label={isBookmarked ? 'Remove Bookmark' : 'Add Bookmark'}
    >
      <svg
        fill={isBookmarked ? 'currentColor' : 'none'}
        stroke="currentColor"
        viewBox="0 0 24 24"
        width="20"
        height="20"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          strokeWidth="2"
          d="M5 5a2 2 0 012-2h10a2 2 0 012 2v16l-7-3.5L5 21V5z"
        />
      </svg>
      <span className="bookmark-count">{count}</span>
    </button>
  );
};
```